ECG Arrhythmia Classification for Microcontroller Deployment: An Inter-Patient Pipeline with Verified C Export and ARM Flash Profiling

Balajee, *Student,* Manipal University Jaipur

***Abstract*—We present a complete, reproducible pipeline for training ECG arrhythmia classifiers deployable to ARM Cortex-M microcontrollers without external inference libraries. Two methodological commitments distinguish this work from the majority of open implementations. First, we follow the standard inter-patient evaluation protocol of de Chazal et al. (2004), training exclusively on patients in dataset DS1 and evaluating exactly once on the held-out patients of DS2; intra-patient cross-validation, which dominates publicly available code, inflates reported accuracy through patient-specific morphology leakage. Second, we verify that the deployed artifact is correct: trained models are exported to dependency-free C via `m2cgen`, cross-compiled with `arm-none-eabi-gcc`, and confirmed to produce bit-identical predictions to their Python counterparts on matched input rows before any hardware deployment is attempted. A 24-feature representation combining morphological waveform statistics, frequency-domain band energies, and RR-interval timing is extracted per beat; the RR features are included specifically to address the known morphology-only limitation for supraventricular ectopic (S) detection. Three model tiers are trained: a 200-tree random forest (*Reference*), a 10-tree random forest with bounded depth (*Edge*), and a single depth-6 decision tree (*Tiny*). On DS2, the Reference model achieves 0.920 accuracy and 0.518 reliable macro-F1 (four-class, excluding the near-empty unclassifiable class); the Tiny model achieves 0.708 accuracy in 3,656 bytes of ARM Cortex-M4 compiled code, while the Edge model achieves 0.830 accuracy in 94,252 bytes. All code, tests, trained model artifacts, and generated C files are publicly available.**

***Index Terms*—ECG arrhythmia, TinyML, edge inference, ARM Cortex-M, inter-patient evaluation, automatic beat classification, AAMI EC57, MIT-BIH**

I. Introduction

CARDIAC arrhythmias affect an estimated 1.5–5% of the global population [1]. Continuous long-term monitoring with wearable Holter monitors or cardiac patches provides substantially richer diagnostic information than a resting ECG, but demands low-power autonomy: a microcontroller-class processor performing local inference rather than streaming raw data to a cloud backend. This constraint motivates the TinyML framing [2]: train the classifier on a workstation, then deploy only the inference path to the resource-limited device.

Two persistent problems limit the practical value of most published ECG classifiers. First, evaluation is frequently performed with intra-patient cross-validation, in which different beats from the *same* patient may appear in both training and test folds. The classifier can then exploit patient-specific baseline QRS morphology rather than learning class-general arrhythmia signatures, producing inflated accuracy figures that do not transfer to unseen patients [3]. Second, the gap between a trained model and a deployed model is typically unverified: code exported to a target language may diverge from the original due to floating-point ordering differences, language-specific rounding, or silent API mismatches.

This paper addresses both problems concurrently. We implement the standard inter-patient evaluation protocol from de Chazal et al. [3] exactly, using their published DS1/DS2 patient partition of the MIT-BIH Arrhythmia Database. We then export trained models to self-contained C code, cross-compile for ARM Cortex-M4, and verify prediction parity between Python and C before reporting any result. The pipeline is fully reproducible: every number in this paper can be regenerated from the public repository by running a single entry-point script.

Contributions

- A complete, single-entry-point training and evaluation pipeline following the de Chazal inter-patient protocol, including RR-interval timing features not present in the original work, with verified per-class precision, recall, and F1 for all five AAMI classes.
- Automated export of trained sklearn classifiers to dependency-free C inference code via `m2cgen`, with a pytest-based C/Python prediction-parity test suite that compiles and runs the C binary against matched input rows.
- Measured ARM Cortex-M4 flash footprints (via `arm-none-eabi-size`) for three model tiers positioned across the size–accuracy tradeoff, demonstrating deployability to commodity embedded targets.

II. Background and Related Work

A. The MIT-BIH Arrhythmia Database

The MIT-BIH Arrhythmia Database [4], [5] contains 48 half-hour ambulatory ECG recordings sampled at 360 Hz. Each recording is annotated beat-by-beat by two cardiologists. Following the AAMI EC57 standard [6], beat annotations are mapped to five classes: Normal (N), Supraventricular ectopic

Manuscript received July 4, 2026. Code and model artifacts: https://github.com/bl4zeh3x/tinyml-explorations.

(S), Ventricular ectopic (V), Fusion (F), and Unclassifiable (Q). The unclassifiable class is drawn almost entirely from paced-beat patients (records 102, 104, 107, 217), which are excluded from both training and test sets in the standard inter-patient protocol, leaving near-zero representation of Q under this evaluation regime.

B. Inter-Patient Evaluation

The pivotal methodological reference is de Chazal et al. [3], who defined a patient-disjoint partition of MIT-BIH: DS1 (22 records for training) and DS2 (22 records for test). This partition is now used as the standard comparison point in the AAMI beat classification literature. Prior work using intra-patient evaluation, including several high-citation CNN and LSTM papers [7], [8], reports substantially higher accuracy because patient-specific QRS morphology partially identifies individuals rather than arrhythmia classes.

C. TinyML and Edge Inference

Deploying inference to MCUs without OS or ML runtime support has been addressed by frameworks including TensorFlow Lite for Microcontrollers (TFLM) [9] and Edge Impulse. These target neural networks and carry a runtime overhead measured in kilobytes. For classical tree-based models, code generators such as `m2cgen` [10] emit plain C with no dependencies beyond the standard math library. The trade-off — smaller and more predictable code at some accuracy cost — is the central design axis of this work.

III. Methods

A. Dataset and Evaluation Protocol

We use the MIT-BIH Arrhythmia Database, downloaded programmatically via the `wfdb` Python package [11]. We implement the de Chazal inter-patient split exactly:

DS1 (train): 101, 106, 108, 109, 112, 114, 115, 116, 118, 119, 122, 124, 201, 203, 205, 207, 208, 209, 215, 220, 223, 230.
DS2 (test): 100, 103, 105, 111, 113, 117, 121, 123, 200, 202, 210, 212, 213, 214, 219, 221, 222, 228, 231, 232, 233, 234.

Models are trained exclusively on DS1 and evaluated exactly once on DS2. No hyperparameter search is conducted; model configurations are fixed in advance based on the deployment size targets described below.

B. Beat Segmentation

Each beat is represented as a 360-sample window (1 second at 360 Hz) centred on the annotated R-peak. Beats with windows extending outside the recording boundary are discarded. Each window is independently z-score normalised (zero mean, unit variance) so that absolute sensor gain differences across leads do not contribute to the classification.

C. Feature Extraction

We extract 24 scalar features per beat, arranged in four groups:

1) Amplitude features (8): R-peak amplitude; global maximum and minimum; peak-to-peak range; Q-wave minimum (40 samples left of R-peak); S-wave minimum (40 samples right of R-peak); R−Q and R−S differences.

2) Statistical features (6): Mean, standard deviation, total signal energy, skewness (non-excess), kurtosis (non-excess), zero-crossing rate.

3) Frequency-domain features (6): FFT band powers in four clinically relevant bands (0–5, 5–15, 15–40, 40–100 Hz), normalised by total spectral power; dominant frequency (first positive-frequency bin excluding DC); spectral centroid.

4) RR-interval timing features (4): Pre-RR interval (time from previous annotated R-peak to current); post-RR interval (time to next annotated R-peak); pre/post ratio; rolling local mean of the previous five pre-RR intervals. These features were added after observing that morphology-only classification produces recall below 10% for the S class: supraventricular ectopic beats are frequently near-identical to Normal beats in waveform shape, but are defined clinically by premature firing — a timing, not a morphology, phenomenon. RR features are computed per-record and reset at record boundaries so that timing information never bridges across patients.

The 24 features reduce each 1440-byte double-precision waveform window (360 samples × 4 bytes) to a 96-byte feature vector (24 floats), matching the compute budget available for real-time extraction on a Cortex-M4 at typical ECG inference cadences.

D. Model Tiers

Three scikit-learn [12] classifiers are trained with `class_weight='balanced'` to address the severe class imbalance (approximately 75% Normal beats in both DS1 and DS2):

- **Reference**: `RandomForestClassifier(n_estimators=200` — maximum-accuracy baseline; not intended for embedded deployment.
- **Edge**: `RandomForestClassifier(n_estimators=10, max_depth=8)` — targeting Cortex-M4 class devices with ≥128 KB flash (e.g. STM32L4 series with 256 KB flash); measured footprint 94,252 bytes.
- **Tiny**: `DecisionTreeClassifier(max_depth=6)` — targeting severely resource-constrained devices such as the ATmega328P (Arduino Uno, 32 KB flash); measured footprint 3,656 bytes.

No cross-validation is used for model selection. DS2 is touched exactly once, after all training decisions are finalised.

E. C Export and Deployment Verification

Each model is exported to C using `m2cgen` [10], which walks the fitted estimator and emits a self-contained `score()` function:

```
void score(double *input, double *output);
```

The function accepts a 24-element feature vector and populates a 5-element class probability array. No dynamic allocation, no external libraries, no floating-point library beyond what the

target FPU natively supports. The Edge and Tiny models are exported; the Reference model is not, as it is not a deployment target.

Parity verification. A pytest suite compiles each generated C file against a C harness using the system `gcc` toolchain and compares the argmax prediction for 10 real feature rows against the corresponding `sklearn` prediction. All parity tests pass for both exported models, confirming that the C inference and the Python model are numerically equivalent on these inputs.

ARM compilation. The verified C files are then cross-compiled for bare-metal ARM Cortex-M4:

```
arm-none-eabi-gcc -mcpu=cortex-m4 \
  -mthumb -std=c99 -O2 -c ecg_<model>.c
arm-none-eabi-size ecg_<model>.o
```

Flash footprints are read from the `.text` segment of the resulting object file.

F. Metrics

We report DS2 accuracy, strict macro-F1 (all five AAMI classes), and *reliable* macro-F1, which excludes classes with fewer than 30 training examples from the average. Only the Q class falls below this threshold (8 training examples in DS1, drawn almost entirely from paced-beat patients that the standard protocol excludes). The excluded set and its threshold are always reported alongside the reliable score; this is not a post-hoc selection but a pre-registered decision motivated by the statistical unreliability of class-level metrics computed from single-digit training support. Full per-class precision, recall, and F1 are reported for all three models.

IV. Results

A. Overall Performance

Table I summarises DS2 held-out performance for all three model tiers together with ARM Cortex-M4 flash footprints.

TABLE I
DS2 Held-Out Performance and ARM Cortex-M4 Flash Footprints

Model	Acc.	F1 (strict)	F1 (reliable)	Nodes	.text
Reference	0.920	0.415	0.518	283,328	—
Edge	0.830	0.409	0.511	∼3,296	94,252
Tiny	0.708	0.324	0.405	121	3,656

Reliable F1 excludes Unclassifiable (Q; 8 DS1 training samples). Flash footprints measured via `arm-none-eabi-gcc -O2 -mcpu=cortex-m4 -mthumb`.

B. Per-Class Analysis

Table II shows precision, recall, and F1 per AAMI class for the Reference model, alongside DS1 and DS2 sample counts.

TABLE II
Reference Model Per-Class Metrics on DS2

Class	Support		P	R	F1
	DS1	DS2			
Normal (N)	45,824	44,218	0.957	0.958	0.958
Supravent. (S)	943	1,836	0.254	0.094	0.138
Ventricular (V)	3,788	3,219	0.842	0.968	0.901
Fusion (F)	414	388	0.053	0.142	0.078
Unclassif. (Q)*	8	7	0.000	0.000	0.000

*Excluded from reliable macro-F1.

C. Model-Tier Error Profiles

A notable qualitative difference emerges between model tiers on the Fusion (F) class: the Edge model achieves 85.1% recall on F at 9.1% precision, while the Reference model achieves 14.2% recall at 5.3% precision. The constrained depth of the Edge model, combined with `class_weight='balanced'` amplifying F's 414-sample presence, produces a broad decision region that captures most true F beats at high false-positive cost. This is not a tuning artefact but a structural consequence of depth-limited tree ensembles on heavily imbalanced classes: shallower models spend more of their limited splits on the imbalance correction, while the Reference model's unconstrained depth allows tighter per-class boundaries. The practical implication is that the deployment tier choice also implicitly selects an error profile, not merely an accuracy level.

V. Discussion

A. The S-Class Gap

Supraventricular ectopic beats achieve F1 of 0.138 on the Reference model despite adequate DS2 support (1,836 test samples). This result is literature-consistent: de Chazal et al. [3] reported positive predictivity around 31% for SVEB using considerably more sophisticated wave-boundary features. The fundamental difficulty is morphological: S-class beats are frequently near-identical to Normal beats in raw waveform shape; what clinically defines them is prematurity, a timing phenomenon our RR features partially address but do not resolve. RR-interval addition improved macro-F1 from 0.316 (morphology only) to 0.415 (with RR features), confirming the features help, while leaving substantial room for future work.

B. Why Classical ML for This Use Case

The MCU constraint provides a principled reason to prefer classical tree-based models over convolutional or recurrent networks. A depth-6 decision tree performs inference as a sequence of scalar comparisons against learned thresholds — an operation that maps to a handful of ARM Thumb-2 instructions with no memory bandwidth requirement beyond the feature vector. CNN-based ECG classifiers achieve stronger absolute performance but require kilobytes of parameter storage, floating-point convolution, and in most published implementations a full ML runtime (TFLM, Caffe, or similar). The accuracy cost of the classical approach, visible in Table I, is the explicit price of removing those dependencies entirely.

The Edge model's 94 KB footprint warrants comment. A 10-tree random forest with `max_depth=8` over 24-dimensional input produces substantially more total nodes than our earlier estimates suggested, reflecting the interaction between tree depth, feature dimensionality, and bootstrap-sample diversity at the given class balance. At 94 KB, the Edge model requires

a target with at least 128 KB flash — common in STM32L4 and nRF52840 series devices but not the smallest Cortex-M0 parts. The Tiny model, at 3,656 bytes, remains compatible with devices as small as 32 KB flash and represents a more aggressive size–accuracy tradeoff: 0.708 accuracy at a flash cost three orders of magnitude smaller than a typical TFLM-based neural network.

C. Limitations

Hardware deployment has not yet been validated; the C models have been compiled and flash-profiled but not executed on physical silicon. Additionally, the pipeline processes one beat at a time from pre-segmented inputs; real-time deployment requires a preceding R-peak detector and ring-buffer management, which are not included in the current pipeline. Finally, the feature extraction step (particularly the FFT for frequency-domain features) has not been benchmarked for latency on an actual Cortex-M4 — only the inference function flash footprint is measured.

VI. Conclusion

We described a reproducible pipeline for inter-patient ECG arrhythmia classification targeting microcontroller deployment. The pipeline implements the de Chazal DS1/DS2 inter-patient split, extracts 24 features combining morphological, spectral, and RR-interval information, trains three classifier tiers spanning 121 to 283,328 tree nodes, exports the two smaller tiers to verified dependency-free C, and measures their ARM Cortex-M4 flash footprints from compiled object files. All code, trained model artifacts, exported C files, and tests are available at https://github.com/bl4zeh3x/tinyml-explorations.

The principal finding is that the N and V AAMI classes are well-handled by classical feature-engineered classifiers under inter-patient evaluation (F1 of 0.958 and 0.901 respectively for the Reference model), while S and F remain difficult in a manner consistent with the broader literature. Combining accurate V-class detection with a 3,656-byte inference function (Tiny model) or a 94,252-byte function (Edge model) opens practical paths to arrhythmia screening on wearable microcontroller platforms of different flash-budget classes.

References

[1] M. Zoni-Berisso, F. Lercari, T. Carazza, and S. Domenicucci, "Epidemiology of atrial fibrillation: European perspective," *Clinical Epidemiology*, vol. 6, pp. 213–220, 2014.

[2] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[3] P. de Chazal, M. O'Dwyer, and R. B. Reilly, "Automatic classification of heartbeats using ECG morphology and heartbeat interval features," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004.

[4] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH arrhythmia database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.

[5] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000.

[6] Association for the Advancement of Medical Instrumentation, *ANSI/AAMI EC57:2012 Testing and Reporting Performance Results of Cardiac Rhythm and ST Segment Measurement Algorithms*, Std., 2012.

[7] S. Kiranyaz, T. Ince, and M. Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016.

[8] U. R. Acharya, S. L. Oh, Y. Hagiwara, J. H. Tan, M. Liu, A. Vidyanathan, and M. M. H. Poo, "A deep convolutional neural network model to classify heartbeats," *Computers in Biology and Medicine*, vol. 89, pp. 389–396, 2017.

[9] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes, "TensorFlow lite micro: Embedded machine learning for TinyML systems," in *Proceedings of Machine Learning and Systems*, vol. 3, 2021, pp. 800–811.

[10] BayesWitnesses, "m2cgen: Transform your ML models into a native code," https://github.com/BayesWitnesses/m2cgen, 2023, gitHub repository.

[11] MIT Laboratory for Computational Physiology, "WFDB python package," https://github.com/MIT-LCP/wfdb-python, 2023, gitHub repository.

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.